



**UNIVERSIDAD
DE GRANADA**

**TRABAJO FIN DE GRADO
INGENIERÍA EN INFORMÁTICA**

Diseño e Implementación de un Simulador de Códigos BCH para el Análisis de Rendimiento en Canales con Ruido

**Desarrollo de un motor aritmético en C++ y entorno de análisis en
Python para la evaluación de la robustez en transmisiones digitales**

Autor

ZhiXiang Zhou Wang

Director

Jesús García Miranda



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE TELECOMUNICACIÓN

—
Granada, Julio de 2026

Diseño e Implementación de un Simulador de Códigos BCH para el Análisis de Rendimiento en Canales con Ruido

ZhiXiang Zhou Wang

Palabras clave: Códigos BCH, Cuerpos de Galois, Simulación, Rendimiento, Teoría de la Información, Python.

Resumen

Este Trabajo Fin de Grado presenta el diseño y la implementación de un simulador integral para códigos de bloque cíclicos BCH (Bose-Chaudhuri-Hocquenghem). El núcleo del proyecto consiste en el desarrollo de un motor aritmético basado en Cuerpos de Galois $GF(2^m)$ que permite realizar operaciones polinomiales eficientes para la codificación y decodificación de datos. Se implementa el algoritmo de Berlekamp-Massey para la localización de errores y la búsqueda de Chien para su corrección. El software desarrollado permite modelar canales de comunicación ruidosos y realizar un análisis comparativo de la tasa de error de bit (BER) frente a diferentes relaciones señal-ruido, proporcionando una herramienta visual y cuantitativa para evaluar la robustez de estos códigos en entornos de transmisión digital.

Design and Implementation of a BCH Code Simulator for Performance Analysis in Noisy Channels

ZhiXiang Zhou Wang

Keywords: BCH Codes, Galois Fields, Simulation, Performance, Information Theory, Python.

Abstract

This Thesis focuses on the design and implementation of a comprehensive simulator for BCH (Bose-Chaudhuri-Hocquenghem) cyclic block codes. The core of the project involves developing an arithmetic engine based on Galois Fields $GF(2^m)$ to perform efficient polynomial operations for data encoding and decoding. The Berlekamp-Massey algorithm is implemented for error localization along with the Chien search for error correction. The developed software enables the modeling of noisy communication channels and conducts a comparative analysis of the Bit Error Rate (BER) against various signal-to-noise ratios, providing a visual and quantitative tool to evaluate the robustness of these codes in digital transmission environments.

Yo, **ZhiXiang Zhou Wang**, alumno de la titulación **Grado en Ingeniería Informática** de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI **12807058Z**, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: ZhiXiang Zhou Wang

Granada, a _____ de Julio de 2026. [cite: 41]

D. **Jesús García Miranda**, Profesor del Área de Álgebra del Departamento de Álgebra de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado *Diseño e Implementación de un Simulador de Códigos BCH para el Análisis de Rendimiento en Canales con Ruido*, ha sido realizado bajo su supervisión por **ZhiXiang Zhou Wang**, y autorizo la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada, a ____ de Julio de 2026.

El director:

Jesús García Miranda

Agradecimientos

En primer lugar, quisiera expresar mi más sincero agradecimiento a mi tutor, D. Jesús García Miranda, por su guía, paciencia y valiosos consejos durante el desarrollo de este trabajo. Su profundo conocimiento del álgebra ha sido fundamental para comprender las bases teóricas de este proyecto.

A mi familia y amigos, por su apoyo incondicional durante estos años de carrera en la ETSIT.

Índice general

1	Introducción	1
1.1	Motivación	1
2	Fundamentos Matemáticos	2
3	Diseño del Simulador	3
3.1	Especificación de Requisitos y Funcionalidades	3
3.1.1	Requisitos Funcionales	3
3.1.2	Requisitos No Funcionales	3
3.2	Diseño del Sistema	4
3.2.1	Arquitectura	4
3.2.2	Flujo de Información y Modelo de Simulación	5
3.2.3	Diseño de Datos y Estructuras	5
3.3	Diseño Detallado de Componentes	6
3.3.1	Subsistema Matemático	6
3.3.2	Subsistema Códec BCH	8
3.3.3	Subsistema de Canal	9
3.3.4	Subsistema de Simulación y Orquestación	9
3.3.5	Decisiones de Diseño y Optimizaciones	10
4	Implementación	11
4.1	Implementación del Cuerpo de Galois	11
4.1.1	Estructura de Datos	11
4.1.2	Inicialización y Precálculo	11
4.1.3	Operaciones Implementadas	12
4.1.4	Gestión de Memoria y Optimizaciones	14
4.1.5	Ejemplo de Uso	14
4.2	Implementación de Polinomios	15
4.2.1	Estructura de datos	15
4.2.2	Constructor y normalización	15
4.2.3	Operaciones implementadas	15
4.2.4	Decisiones de diseño	16
4.3	Implementación de PackedBit	17
4.3.1	Estructura de datos	17

4.3.2	Constructor y conversión desde vector	17
4.3.3	Operaciones implementadas	17
4.3.4	Decisiones de diseño	19
4.3.5	Relación con BCH_Codec	19
4.4	Implementación de BCH_Codec	19
4.4.1	Estructura de datos	19
4.4.2	Constructor y cálculo del generador	20
4.4.3	Codificación sistemática (versión empaquetada)	20
4.4.4	Detección de errores mediante síndromes	21
4.4.5	Localización de errores con Berlekamp-Massey	22
4.4.6	Decodificación	23
4.4.7	Comentarios de diseño	24
4.5	Implementación del Canal de Ruido	25
4.5.1	Interfaz pública	25
4.5.2	Constructor	25
4.5.3	Canal Binario Simétrico (BSC)	25
4.5.4	Canal AWGN	26
4.5.5	Decisión dura sobre AWGN	26
4.5.6	Decisiones de diseño y uso	26
4.6	Arquitectura del Simulador y Motor de Ejecución	27
4.6.1	Estructuras de Configuración y Resultados	27
4.6.2	Motor de Simulación y Paralelización (OpenMP)	27
4.6.3	Decisiones de Diseño Adicionales	29
5	Resultados y Análisis	30
5.1	Metodología Experimental	30
5.2	Impacto de la capacidad de corrección t para un valor de m fijo	31
5.2.1	Saturación del decodificador en entornos de alto ruido	32
5.2.2	Efecto umbral y coste de la redundancia	32
5.2.3	Diferencia entre BER y CWER	33
5.2.4	Conclusión	33
5.3	Comparación de rendimiento entre códigos con tasa de código similar	33
5.3.1	Análisis del rendimiento en función de la longitud de bloque	34
5.3.2	Análisis de sensibilidad según la capacidad correctora	34
5.3.3	Discusión global y criterios de diseño	35
	Bibliografía	37

Índice de figuras

3.1	Diagrama de clases UML general del simulador BCH.	4
3.2	Flujo de información paralelo para simulación de Monte Carlo.	5
3.3	Diagrama UML del subsistema matemático.	7
3.4	Diagrama UML del subsistema Códec BCH.	8
3.5	Diagrama UML del subsistema de simulación y orquestación.	10
5.1	Estudio comparativo para $m = 7$: Tasa de Error de Bit (BER, izquierda) y Tasa de Error de Palabra (CWER, derecha) para $t \in \{1, 3, 7, 15\}$	32
5.2	Estudio comparativo de la BER para agrupaciones de tasa similar: escena- rios de tasa alta (izquierda), media (centro) y baja (derecha).	34
5.3	Estudio de sensibilidad de la tasa de código agrupado por grado de exten- sión: $m = 5$ (izquierda), $m = 7$ (centro) y $m = 11$ (derecha).	36

Índice de tablas

5.1	Parámetros de los códigos BCH para $m = 7$ ($n = 127$).	31
5.2	Agrupación simétrica de códigos BCH bajo criterios de tasa de código similar.	33

Capítulo 1

Introducción

1.1. Motivación

Capítulo 2

Fundamentos Matemáticos

Capítulo 3

Diseño del Simulador

En este capítulo se presenta la arquitectura del simulador de códigos BCH desarrollado, detallando las decisiones de diseño, la organización de los componentes y el flujo de información.

3.1. Especificación de Requisitos y Funcionalidades

3.1.1. Requisitos Funcionales

- **RF1.** Aritmética completa en el cuerpo finito $GF(2^m)$ mediante tablas log/antilog.
- **RF2.** Configuración flexible de parámetros (m, t) con cálculo automático de n y k .
- **RF3.** Codificación sistemática mediante construcción del polinomio generador.
- **RF4.** Simulación de canales BSC y AWGN con modulación BPSK (decisión dura).
- **RF5.** Decodificación algebraica completa (síndromes, Berlekamp-Massey y búsqueda de Chien).
- **RF6.** Soporte para procesamiento de ficheros reales.
- **RF7.** Recolección y exportación de métricas (BER, BER uncoded, CWER y tiempos).

3.1.2. Requisitos No Funcionales

Los requisitos no funcionales definen las restricciones y atributos de calidad del sistema:

- **RNF1. Rendimiento:** Procesar millones de palabras código en tiempos razonables mediante optimizaciones de bajo nivel y paralelización.
- **RNF2. Modularidad y Reusabilidad:** Separación clara entre álgebra, códec y simulación.

- **RNF3. Reproducibilidad:** Resultados deterministas cuando se fija una semilla.
- **RNF4. Mantenibilidad:** Código bien documentado y orientado a objetos.
- **RNF5. Limitaciones aceptadas:** Soporte hasta $m = 15$, solo decodificación *hard-decision*.

3.2. Diseño del Sistema

Se ha diseñado el software bajo un enfoque modular, facilitando la validación independiente de la lógica matemática y de la lógica de simulación.

3.2.1. Arquitectura

La estructura se divide en tres niveles de abstracción:

1. **Capa de Álgebra:** Contiene las abstracciones matemáticas de elementos del Cuerpo y polinomios.
2. **Capa de Aplicación (BCH):** Contiene el protocolo de codificación y decodificación, utilizando los servicios de la capa de álgebra.
3. **Capa de Simulación:** Contiene el proceso de generación de ruido estadístico, la paralelización por hilos y la recolección de métricas de rendimiento.

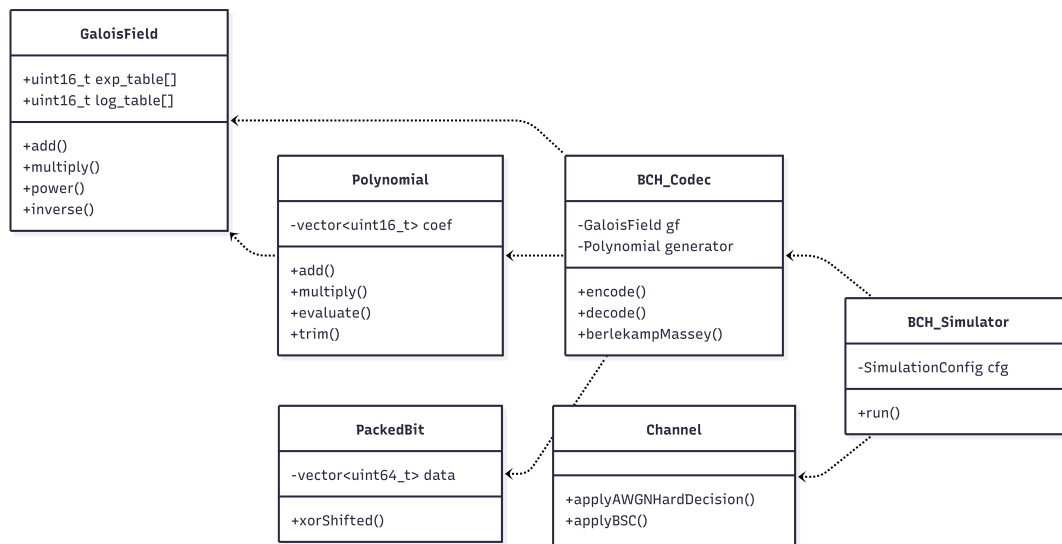


Figura 3.1: Diagrama de clases UML general del simulador BCH.

3.2.2. Flujo de Información y Modelo de Simulación

El simulador sigue un esquema de procesamiento paralelo por palabra código, aprovechando la independencia entre simulaciones Monte Carlo.

A diferencia de un enfoque secuencial, el diseño distribuye la carga de trabajo mediante **OpenMP**, donde cada hilo ejecuta de forma autónoma el ciclo completo: generación del mensaje $\rightarrow$ codificación $\rightarrow$ paso por el canal $\rightarrow$ decodificación $\rightarrow$ recolección de errores.

En la Figura 3.2 se muestra este flujo paralelo. La configuración global (**SimulationConfig**) inicializa las tablas estáticas del Cuerpo de Galois una única vez. Posteriormente, el **BCH_Simulator** lanza un pool de hilos que ejecutan de manera independiente el pipeline de simulación. Los resultados se sincronizan al final para calcular las métricas agregadas (BER, CWER, tiempos, etc.).

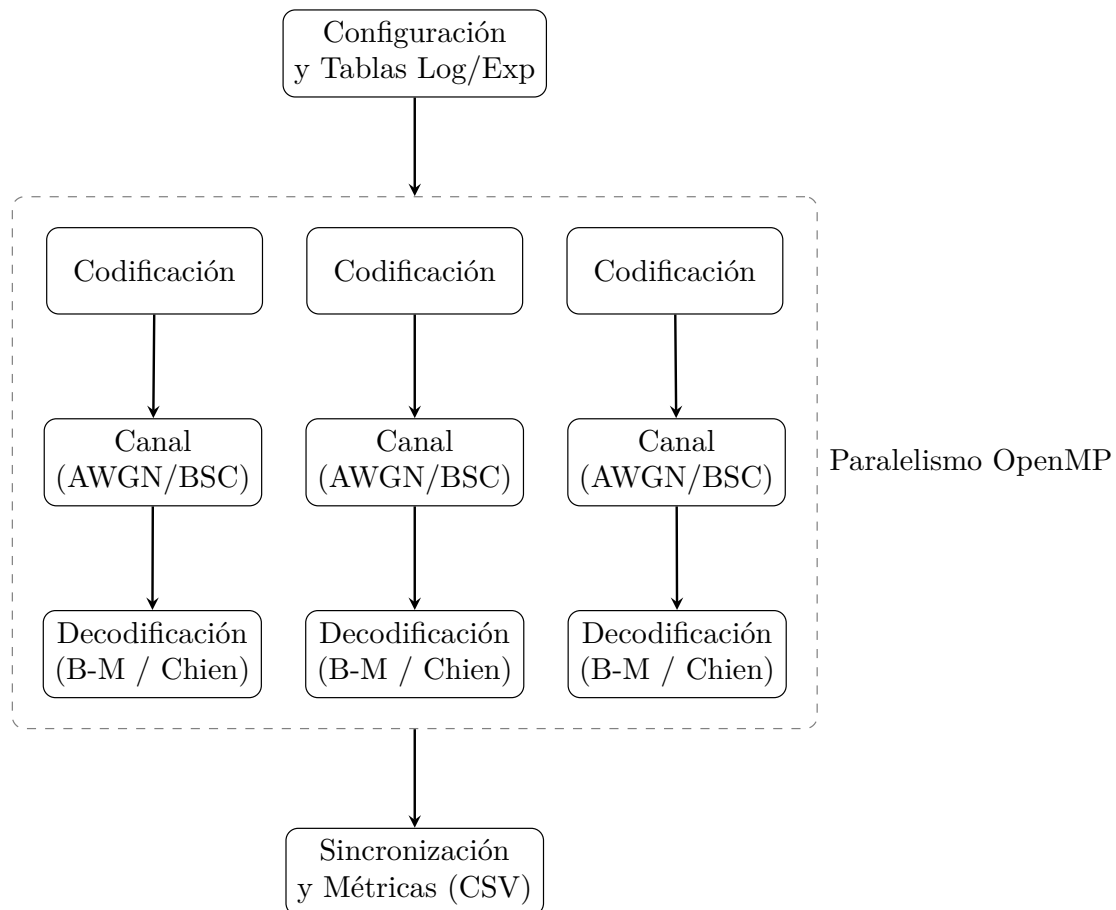


Figura 3.2: Flujo de información paralelo para simulación de Monte Carlo.

3.2.3. Diseño de Datos y Estructuras

Para conseguir un buen equilibrio entre rendimiento y claridad del código, se han tomado las siguientes decisiones clave de representación de datos:

- **Elementos del cuerpo y coeficientes polinómicos:** Utilización de `uint16_t` (suficiente para $m \leq 15$) para mejorar la localidad de memoria y el uso de caché.
- **Optimización de operaciones bit a bit:** La clase `PackedBit` permite realizar desplazamientos y XOR en bloques de 64 bits, fundamental para acelerar la división polinómica en la codificación sistemática.
- **Cuerpo de Galois:** Tablas `exp_table` y `log_table` declaradas como **estáticas**, calculándose una sola vez por valor de m .
- **Polinomios:** Representados mediante `std::vector<uint16_t>` con método `trim()` para eliminar coeficientes nulos de alto grado.
- **Síndromes y decodificación:** Optimización del cálculo de síndromes pares ($S_{2i} = S_i^2$) y uso de vectores auxiliares en Berlekamp-Massey y búsqueda de Chien.
- **Generación de ruido:** Cada instancia de `Channel` utiliza su propio generador `std::mt19937` para garantizar independencia entre hilos.

3.3. Diseño Detallado de Componentes

A continuación se describen los componentes principales del simulador, justificando las decisiones de diseño más relevantes desde el punto de vista matemático, algorítmico y de rendimiento.

3.3.1. Subsistema Matemático

El subsistema matemático constituye la base algebraica del simulador y está formado por las clases `GaloisField`, `Polynomial` y `PackedBit`.

A continuación se describen las clases principales que lo componen:

Clase `GaloisField`

La clase `GaloisField` constituye el fundamento algebraico del simulador. Implementa la aritmética completa en el cuerpo finito $GF(2^m)$ mediante representación logarítmica/exponencial (tablas log/antilog).

- **Construcción:** Recibe el grado m y un polinomio primitivo irreducible. Calcula y almacena en tablas estáticas (`exp_table` y `log_table`) las potencias de un elemento primitivo.
- **Operaciones implementadas:** suma (XOR), multiplicación, división, inversa y potenciación, todas optimizadas mediante acceso a tablas.
- **Decisiones de diseño:**

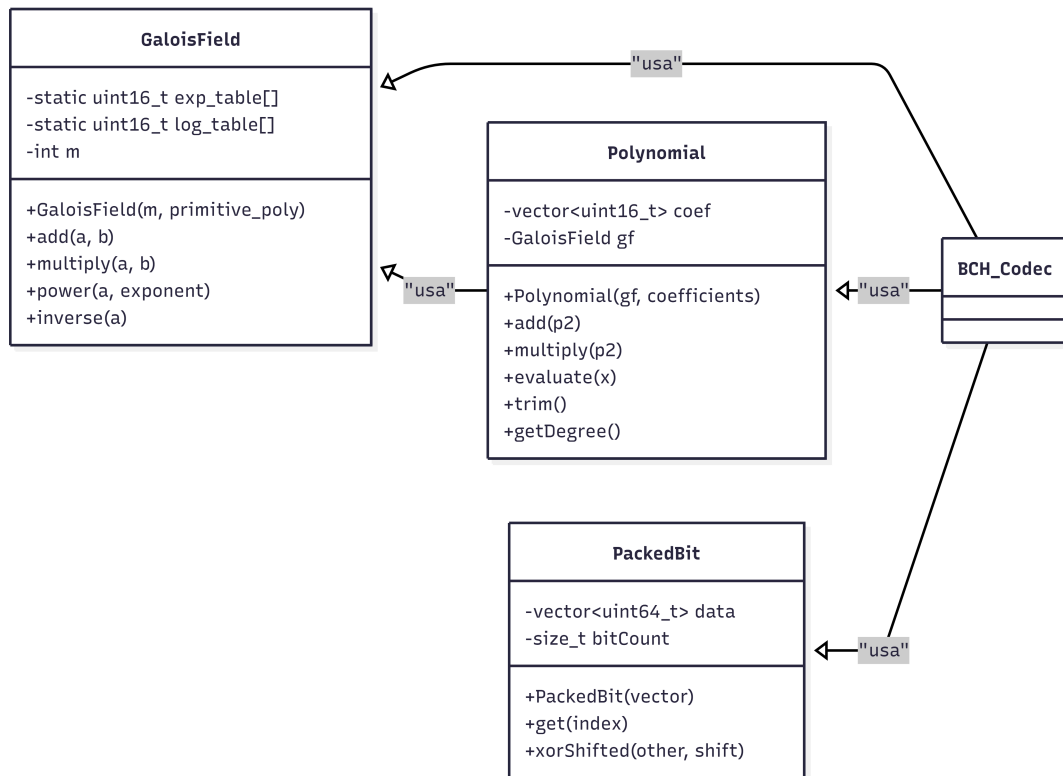


Figura 3.3: Diagrama UML del subsistema matemático.

- Tablas estáticas compartidas entre todas las instancias (se calculan una única vez por valor de m).
- Soporte hasta $m = 15$ (límite práctico impuesto por `uint16_t`).
- Polinomios primitivos predefinidos para cada m soportado.

Clase Polynomial

Representa polinomios sobre $GF(2^m)$. Los coeficientes se almacenan en un `vector<uint16_t>`, donde el índice corresponde al grado del monomio.

- Métodos principales: `add()`, `multiply()`, `evaluate()` y `trim()`.
- El método `trim()` elimina coeficientes nulos de mayor grado, manteniendo el tamaño mínimo del vector.
- Uso principal: construcción del polinomio generador y evaluación de síndromes.

Clase PackedBit

Estructura optimizada para operaciones a nivel de bits. Almacena los bits empaquetados en bloques de 64 bits (`uint64_t`).

Su operación más crítica es `xorShifted()`, utilizada en la división polinómica durante la codificación sistemática. Esta optimización aporta una mejora significativa de rendimiento respecto a la implementación vectorial original.

3.3.2. Subsistema Códec BCH

El núcleo del simulador se encuentra en la clase `BCH_Codec`, que implementa toda la lógica específica de los códigos BCH (codificación sistemática y decodificación algebraica).

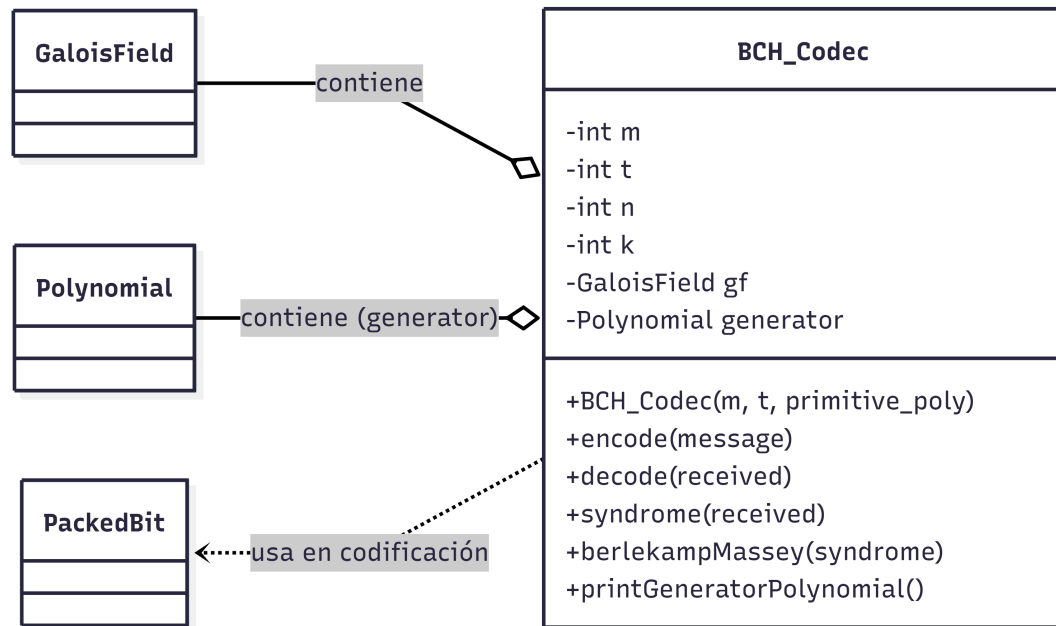


Figura 3.4: Diagrama UML del subsistema Códec BCH.

A continuación se describen los detalles de su implementación:

Clase `BCH_Codec`

Clase central que encapsula toda la lógica específica de los códigos BCH.

■ Inicialización:

- Calcula $n = 2^m - 1$.
- Construye el polinomio generador $g(x)$ multiplicando los polinomios mínimos correspondientes a las raíces $\alpha^1, \alpha^2, \dots, \alpha^{2t}$.
- Calcula automáticamente $k = n - \deg(g(x))$.

■ Codificación sistemática: Se implementaron y compararon dos estrategias:

- Versión basada en `PackedBit` (seleccionada finalmente por su superior rendimiento).

- Versión basada en registro de desplazamiento lineal (LFSR).
- **Decodificación algebraica:**
 - Cálculo de síndromes con optimización para índices pares ($S_{2i} = S_i^2$).
 - Algoritmo de **Berlekamp-Massey** optimizado (trabajando directamente con vectores para reducir el *overhead* de objetos `Polynomial`).
 - Búsqueda de Chien para localizar y corregir hasta t errores por palabra código.

3.3.3. Subsistema de Canal

Clase `Channel`

Implementa los dos modelos de canal utilizados en las simulaciones:

- **Canal Binario Simétrico (BSC):** Invierte cada bit de forma independiente con probabilidad p .
- **Canal AWGN con modulación BPSK y decisión dura:**
 - Mapea los bits según la convención $0 \mapsto +1$ y $1 \mapsto -1$.
 - Añade ruido gaussiano blanco aditivo con varianza calculada en función de E_s/N_0 .
 - Realiza decisión dura sobre la señal recibida.

3.3.4. Subsistema de Simulación y Orquestación

Este subsistema es el responsable de coordinar todo el proceso de simulación Monte Carlo, gestionando la configuración, la paralelización y la recolección de resultados.

A continuación se detallan las clases principales:

Clase `BCH_Simulator`

Es el orquestador principal del simulador. Gestiona:

- La configuración de la simulación (`SimulationConfig`).
- La ejecución de simulaciones Monte Carlo para diferentes valores de E_b/N_0 .
- La paralelización con **OpenMP** (planificación dinámica).
- Criterios de parada inteligentes: mínimo número de *codewords* erróneas o máximo número de iteraciones.
- La recolección de métricas (BER, BER uncoded, CWER y tiempos de codificación/decodificación).
- La exportación de resultados a formato CSV.

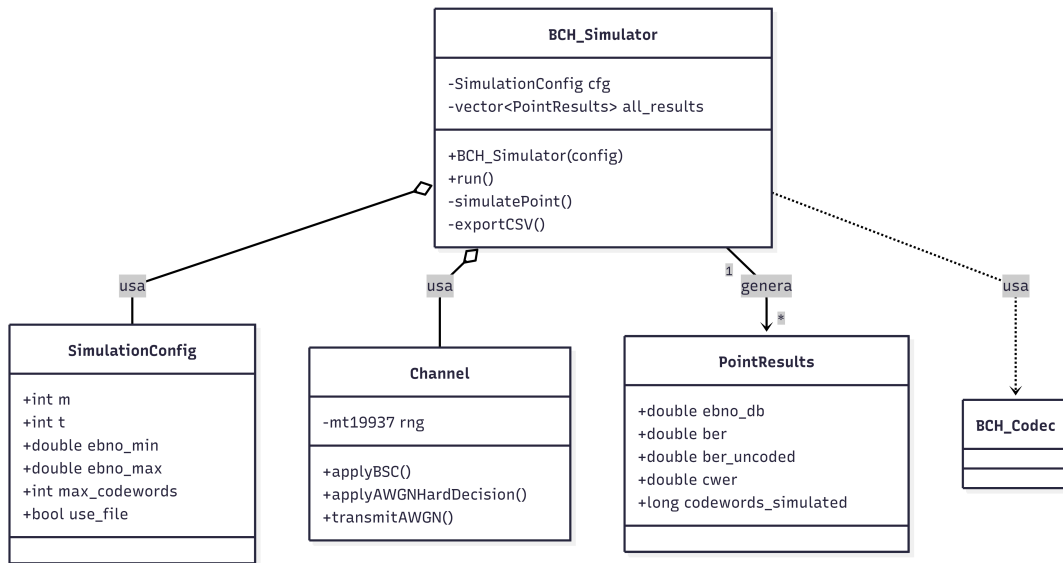


Figura 3.5: Diagrama UML del subsistema de simulación y orquestación.

3.3.5. Decisiones de Diseño y Optimizaciones

Las principales decisiones de diseño estuvieron orientadas a maximizar el rendimiento en simulaciones Monte Carlo:

- Elección de C++ en lugar de Python o MATLAB por su cercanía al hardware y control de bajo nivel.
- Uso intensivo de optimizaciones (**PackedBit**, tablas estáticas).
- Paralelización con OpenMP en lugar de múltiples procesos para reducir overhead.
- Decodificación *hard-decision* como compromiso entre complejidad y representatividad.

Estas decisiones permitieron ejecutar simulaciones con millones de palabras código en tiempos razonables, algo fundamental para obtener curvas BER/CWER confiables.

Capítulo 4

Implementación

En este capítulo se presentan los aspectos más relevantes de la implementación del simulador en C++17.

4.1. Implementación del Cuerpo de Galois

4.1.1. Estructura de Datos

Atributos Estáticos

La clase `GaloisField` utiliza atributos estáticos para compartir las tablas de precálculo entre todas las instancias del mismo cuerpo:

```
1 static vector<uint16_t> log_table;      // Tabla de logaritmos
2 static vector<uint16_t> exp_table;      // Tabla de exponentes
3 static int initialized_m;               // m del ultimo cuerpo inicializado
```

Esta estrategia reduce el consumo de memoria y las operaciones redundantes cuando se crean múltiples instancias del mismo cuerpo.

Atributos de Instancia

```
1 int m;                                // Grado del cuerpo
2 int size;                              // Numero de elementos (2^m)
3 int primitive_poly;                     // Polinomio primitivo
```

4.1.2. Inicialización y Precálculo

El constructor inicializa el cuerpo y genera las tablas de logaritmos y exponentes:

```

1  GaloisField::GaloisField(int m, int primitive_poly)
2      : m(m), size(0), primitive_poly(primitive_poly) {
3      if (m < 1 || m > 15) {
4          throw std::invalid_argument("m debe estar entre 1 y 15");
5      }
6      if (primitive_poly <= 0) {
7          throw std::invalid_argument(
8              "El polinomio primitivo debe ser positivo");
9      }
10
11     size = 1 << m; // size = 2^m
12
13     // Solo se computa si es un nuevo cuerpo
14     if (initialized_m != m) {
15         log_table.assign(size, 0);
16         exp_table.assign(2 * size, 0);
17
18         int val = 1;
19         for (int i = 0; i < size - 1; i++) {
20             exp_table[i] = val;
21             log_table[val] = i;
22             val <<= 1;
23             if (val & size) val ^= primitive_poly;
24         }
25
26         // Duplicar la tabla de exponentes para evitar modulo
27         for (int i = size - 1; i < (size - 1) * 2; i++) {
28             exp_table[i] = exp_table[i - (size - 1)];
29         }
30         initialized_m = m;
31     }
32 }

```

4.1.3. Operaciones Implementadas

Adición

```

1  uint16_t GaloisField::add(uint16_t a, uint16_t b) const {
2      return a ^ b;
3  }

```

Complejidad: $O(1)$. La adición es una operación XOR.

Multiplicación

```

1  uint16_t GaloisField::multiply(uint16_t a, uint16_t b) const {
2      assert(a < size && b < size &&
3          "Elemento fuera de rango");
4      if (a == 0 || b == 0) return 0;
5      return exp_table[log_table[a] + log_table[b]];
6  }

```

Complejidad: $O(1)$ con tablas.

Fundamento Matemático: Se utiliza la propiedad de logaritmos en cuerpos finitos:

$$a \cdot b = \alpha^{\log(a)} \cdot \alpha^{\log(b)} = \alpha^{\log(a)+\log(b)} \quad (4.1)$$

Debido a que la tabla de exponentes se duplica, se evita la operación módulo $2^m - 1$ en cada multiplicación.

División

```

1 uint16_t GaloisField::divide(uint16_t a, uint16_t b) const {
2     assert(a < size && b < size &&
3         "Elemento fuera de rango");
4     assert(b != 0 && "Division por cero");
5     if (a == 0) return 0;
6     return exp_table[log_table[a] - log_table[b] + (size - 1)];
7 }
```

Complejidad: $O(1)$.

Fundamento Matemático:

$$\frac{a}{b} = \alpha^{\log(a)-\log(b)} \quad (4.2)$$

Se suma $(2^m - 1)$ para garantizar un índice positivo, aprovechando la duplicación:
 $\alpha^{\log(a)-\log(b)+(2^m-1)} = \alpha^{\log(a)-\log(b)}$ en $\mathbb{GF}(2^m)$.

Potencia

```

1 uint16_t GaloisField::power(uint16_t a, int e) const {
2     assert(a < size && "Elemento fuera de rango");
3     assert(e >= 0 && "Exponente negativo no soportado");
4     if (a == 0) return 0;
5     if (e == 0) return 1;
6     int exp = log_table[a] * e % (size - 1);
7     return exp_table[exp];
8 }
```

Complejidad: $O(1)$.

Fundamento Matemático:

$$a^e = (\alpha^{\log(a)})^e = \alpha^{e \cdot \log(a) \bmod (2^m-1)} \quad (4.3)$$

Inverso Multiplicativo

```

1 uint16_t GaloisField::inverse(uint16_t a) const {
2     assert(a < size && "Elemento fuera de rango");
3     assert(a != 0 && "No se puede invertir cero");
4     return exp_table[(size - 1) - log_table[a]];
5 }

```

Complejidad: $O(1)$.

Fundamento Matemático: El inverso multiplicativo de $a = \alpha^{\log(a)}$ es:

$$a^{-1} = \alpha^{-(\log(a))} = \alpha^{(2^m-1)-\log(a)} \quad (4.4)$$

garantizando que $a \cdot a^{-1} = \alpha^{2^m-1} = 1$.

4.1.4. Gestión de Memoria y Optimizaciones

El uso de tablas estáticas proporciona las siguientes ventajas:

- **Precálculo único:** Si múltiples instancias utilizan el mismo m , las tablas se computan solo una vez
- **Acceso $O(1)$:** Todas las operaciones se reducen a búsquedas en tabla
- **Validación verificada:** El cuerpo `initialized_m` garantiza consistencia
- **Complejidad espacial:** Para $\mathbb{GF}(2^m)$ aproximadamente $3 \times 2^{m+1}$ bytes. Para $m = 15$ (máximo soportado): ≈ 192 KB

Para las operaciones aritméticas críticas (multiply, divide, power, inverse), la validación de los operandos se realiza mediante `assert`. Esta decisión de diseño se fundamenta en el rendimiento: al compilar el simulador en modo `NDEBUG`, el preprocesador elimina las aserciones, asegurando que no exista ninguna penalización (overhead) en tiempo de ejecución durante las simulaciones intensivas, mientras que mantiene la seguridad durante la fase de desarrollo y depuración.

4.1.5. Ejemplo de Uso

```

1 // Crear un cuerpo GF(2^4) con polinomio primitivo 0x13
2 GaloisField gf(4, 0x13);
3
4 // Operaciones
5 uint16_t a = 5; // 0b0101 en binario
6 uint16_t b = 7; // 0b0111 en binario
7
8 uint16_t suma = gf.add(a, b); // 5 XOR 7 = 2 (0b0010)
9 uint16_t producto = gf.multiply(a, b); // 5 * 7 = 8 (0b1000)
10 uint16_t cociente = gf.divide(a, b); // 5 / 7 = 13 (0b1101)
11 uint16_t inverso = gf.inverse(b); // 7^-1 = 6 (0b0110)

```

```
12 uint16_t potencia = gf.power(a, 3);    //  $5^3 = 10$  (0b1010)
```

4.2. Implementación de Polinomios

4.2.1. Estructura de datos

La clase `Polynomial` representa polinomios con coeficientes en $\mathbb{GF}(2^m)$ usando:

```
1 // coef[0] + coef[1]x + coef[2]x^2 + ...
2 vector<uint16_t> coef;
3 GaloisField gf; // cuerpo asociado
```

`trim()` mantiene la representación eliminando ceros finales.

4.2.2. Constructor y normalización

```
1 Polynomial::Polynomial(const GaloisField &galoisField, const vector<uint16_t> &coefficients)
2     : coef(coefficients), gf(galoisField) {
3     trim();
4 }
```

4.2.3. Operaciones implementadas

Acceso y modificación

```
1 int Polynomial::getDegree() const { /* devuelve coef.size()-1 o -1 si vacío */ }
2 uint16_t Polynomial::getCoef(int degree) const { /* devuelve 0 si fuera de rango */ }
3 int Polynomial::setCoef(int degree, uint16_t value) { /* redimensiona y trim */ }
```

Suma

Suma coeficiente a coeficiente usando `gf.add`:

```
1 Polynomial Polynomial::add(const Polynomial &p2) const {
2     std::vector<uint16_t> result_coef(std::max(coef.size(), p2.coef.size()), 0);
3     for (size_t i = 0; i < result_coef.size(); i++) {
4         uint16_t c1 = (i < coef.size()) ? coef[i] : 0;
5         uint16_t c2 = (i < p2.coef.size()) ? p2.coef[i] : 0;
6         result_coef[i] = gf.add(c1, c2);
7     }
8     return Polynomial(gf, result_coef);
9 }
```

Complejidad: $O(n)$ donde $n = \max(\deg(p1), \deg(p2))$.

Multiplicación

Multiplicación por el método directo (coste cuadrático):

```

1 Polynomial Polynomial::multiply(const Polynomial &p2) const{
2     if (coef.empty() || p2.coef.empty()) return Polynomial(gf, {});
3     std::vector<uint16_t> result_coef(coef.size() + p2.coef.size() - 1, 0);
4     for(size_t i = 0; i < coef.size(); i++){
5         for(size_t j=0; j<p2.coef.size(); j++){
6             result_coef[i+j] = gf.add(result_coef[i+j], gf.multiply(coef[i], p2.coef[j]));
7         }
8     }
9     return Polynomial(gf, result_coef);
10 }
```

Complejidad: $O(n \cdot m)$ para grados n, m .

Evaluación (Horner)

```

1 uint16_t Polynomial::evaluate(uint16_t x) {
2     if (coef.empty()) return 0;
3     if (x == 0) return getCoef(0);
4     uint16_t res = 0;
5     for (int i = getDegree(); i >= 0; i--) {
6         res = gf.add(gf.multiply(res, x), getCoef(i));
7     }
8     return res;
9 }
```

Trim y utilidades

```

1 void Polynomial::trim() {
2     while (!coef.empty() && coef.back() == 0) coef.pop_back();
3 }
```

4.2.4. Decisiones de diseño

- Algoritmo de multiplicación simple: suficiente para los grados usados en este proyecto; alternativas (Karatsuba) solo si se requiere rendimiento en polinomios grandes.
- Uso de `uint16_t` para coeficientes acorde a `GaloisField` actual ($m \leq 15$).

4.3. Implementación de PackedBit

4.3.1. Estructura de datos

La clase `PackedBit` empaqueta una secuencia de bits lógicos dentro de un vector de bloques de 64 bits (`vector<uint64_t>`). Esta representación reduce el coste de memoria frente a almacenar cada bit como un entero independiente y permite aplicar XOR y desplazamientos a nivel de palabra completa.

En el contexto del codificador BCH, `PackedBit` se usa para representar tanto el mensaje extendido como el polinomio generador en forma binaria, acelerando la división polinómica por XOR desplazado.

4.3.2. Constructor y conversión desde vector

La construcción parte de un `std::vector<uint16_t>` en el que cada entrada se interpreta como un bit lógico. El constructor calcula el número de bloques necesarios, reserva memoria y fija cada bit en la posición correspondiente.

```
1 PackedBit(const std::vector<uint16_t> &original){
2     bitCount = original.size();
3     size_t blockCount = (bitCount + 63) / 64;
4     data.resize(blockCount, 0);
5     for (size_t i=0; i<bitCount; ++i){
6         if (original[i]){
7             data[ i/64 ] |= (1ULL << (i % 64));
8         }
9     }
10 }
```

4.3.3. Operaciones implementadas

Acceso y modificación de bits

```
1 uint16_t get(size_t index) const;
2 void set(size_t index, uint16_t value);
```

`get()` devuelve 0 cuando el índice está fuera de rango, y `set()` ignora escrituras fuera del tamaño lógico del objeto.

Conversión a vector

Para facilitar la interoperabilidad de la estructura `PackedBit` con otras etapas del simulador (como el modulador o la simulación del canal con ruido), la clase proporciona el método `toVector()`, el cual expande los bits empaquetados en un `std::vector<uint16_t>`.

```
1  std::vector<uint16_t> toVector(){
2      std::vector<uint16_t> res(bitCount, 0);
3      size_t fullBlocks = bitCount / 64;
4      uint64_t blockData;
5      for(size_t block=0; block<fullBlocks; block++){
6          blockData = data[block];
7          for(size_t offset=0; offset<64; offset++){
8              res[block*64 + offset] = (blockData >> offset) & 1ULL;
9          }
10     }
11     size_t remainingBits = bitCount % 64;
12     if (remainingBits > 0) {
13         uint64_t blockData = data[fullBlocks];
14         for(size_t offset=0; offset<remainingBits; offset++){
15             res[fullBlocks*64 + offset] = (blockData >> offset) & 1ULL;
16         }
17     }
18     return res;
19 }
```

Desplazamiento y XOR combinado

La operación `xorShifted()` es la más importante para el codificador BCH, porque permite simular la resta polinómica mediante XOR del generador desplazado.

```
1  void xorShifted(const PackedBit &gx, size_t shift_bits) {
2      if (shift_bits >= bitCount || gx.data.empty()) return;
3
4      size_t block_shift = shift_bits / 64;
5      size_t bit_shift = shift_bits % 64;
6      uint64_t carry = 0, next_carry = 0;
7
8      size_t dst;
9      for (size_t i = 0; i < gx.data.size(); ++i) {
10         dst = i + block_shift;
11         if (dst >= data.size()) break;
12
13         uint64_t word = gx.data[i];
14         next_carry = (bit_shift == 0) ? 0 : (word >> (64 - bit_shift));
15         data[dst] ^= (word << bit_shift) | carry;
16         carry = next_carry;
17     }
18
19     dst = gx.data.size() + block_shift;
20     if (carry && dst < data.size()) {
21         data[dst] ^= carry;
22     }
23
24     if (bitCount % 64 != 0 && !data.empty()) {
25         uint64_t keep_mask = (1ULL << (bitCount % 64)) - 1ULL;
26         data.back() &= keep_mask;
27     }
28 }
```

4.3.4. Decisiones de diseño

- Se usa `uint64_t` como unidad base para aprovechar operaciones de la CPU.
- `bitCount` conserva el tamaño lógico original del conjunto de bits.
- Se prioriza la velocidad de operaciones binarias frente a una abstracción más general.
- La versión optimizada de `toVector()` reduce el coste de conversión cuando el codificador necesita recuperar los bits residuales.

4.3.5. Relación con BCH_Codec

`PackedBit` no es solo una estructura auxiliar, sino la representación binaria que permite que `BCH_Codec` realice la división del mensaje por el polinomio generador mediante desplazamientos y XOR sobre bloques de 64 bits. En otras palabras, convierte una operación algebraica sobre polinomios en una secuencia de operaciones bit a bit mucho más eficiente.

4.4. Implementación de BCH_Codec

En esta sección se presenta la implementación completa de la clase `BCH_Codec`, que concentra tanto la generación del polinomio generador como la codificación, la detección de errores y la corrección de bloques recibidos. A diferencia de las clases anteriores, aquí no se implementa una sola operación aislada, sino el flujo completo del codificador/decodificador BCH.

4.4.1. Estructura de datos

Los atributos relevantes de la clase `BCH_Codec` son:

```
1 int m;           // Grado del cuerpo
2 int t;           // Capacidad de correccion
3 int n;           // Longitud del codigo (2^m - 1)
4 int k;           // Longitud del mensaje (n - grado(generator))
5 GaloisField gf;  // cuerpo asociado
6 Polynomial generator; // Polinomio generador
```

Además, la clase mantiene un método privado `computeGeneratorPolynomial()` encargado de construir el polinomio generador a partir de los polinomios mínimos necesarios para el BCH elegido.

4.4.2. Constructor y cálculo del generador

El constructor comprueba que $1 \leq m \leq 15$ y $1 \leq 2t < n$. Una vez validados los parámetros, inicializa el cuerpo de Galois, calcula $n = 2^m - 1$ y determina k a partir del grado del polinomio generador. La construcción del generador se delega a `computeGeneratorPolynomial()`, que recorre las raíces conjugadas y acumula los polinomios mínimos correspondientes.

```

1  BCH_Codec::BCH_Codec(int m, int t, int primitive_poly)
2      : m(m), t(t), gf(m, primitive_poly), generator(gf, {}) {
3      if (m < 1 || m > 15) {
4          throw std::invalid_argument("Invalid BCH parameters: m must be between 1 and 15");
5      }
6
7      n = (1 << m) - 1;
8
9      if (t <= 0 || 2LL * t >= n) {
10         throw std::invalid_argument("Invalid BCH parameters: t must satisfy 1 <= 2t < n");
11     }
12
13     computeGeneratorPolynomial();
14     k = n - generator.getDegree();
15 }

```

```

1  void BCH_Codec::computeGeneratorPolynomial() {
2      generator = Polynomial(gf, std::vector<uint16_t>{1});
3      std::unordered_set<uint16_t> processed_exp;
4
5      for (uint16_t i = 1; i <= 2 * t; i++) {
6          if (processed_exp.count(i)) continue;
7
8          Polynomial mi(gf, {1});
9          uint16_t currExp = i;
10
11         do {
12             uint16_t root = gf.power(2, currExp);
13             mi = mi * Polynomial(gf, {root, 1});
14             processed_exp.insert(currExp);
15             currExp = (currExp * 2) % n;
16         } while (currExp != i);
17
18         generator = generator * mi;
19     }
20 }

```

4.4.3. Codificación sistemática (versión empaquetada)

La versión optimizada utilizada en el simulador convierte el mensaje y el generador en `PackedBit` y realiza la división polinómica mediante XOR desplazado. Esta codificación es sistemática: los bits de información se conservan y los bits de paridad se añaden al principio de la palabra de código.

Las etapas principales son:

1. Extender el mensaje añadiendo g ceros a la izquierda, donde $g = \deg(\text{generator})$.
2. Construir `PackedBit` para el vector extendido y para el polinomio generador.
3. Iterar desde el bit más significativo; si el bit es 1, aplicar `xorShifted` con el generador alineado.
4. Convertir el buffer resultante a vector y extraer los primeros g bits como paridad. Concatenar paridad y mensaje.

```

1  std::vector<uint16_t> BCH_Codec::encode(const std::vector<uint16_t>& message) {
2      if (message.size() != static_cast<size_t>(k))
3          throw std::invalid_argument("Message length must be exactly k");
4
5      int g = generator.getDegree();
6
7      // 1. Mensaje extendido:  $m(x) * x^g$ 
8      std::vector<uint16_t> buffer(g, 0);
9      buffer.insert(buffer.end(), message.begin(), message.end());
10     PackedBit binaryBuffer(buffer);
11
12     // 2. Generador en forma binaria
13     std::vector<uint16_t> gen_bits(g+1, 0);
14     for (int deg = 0; deg <= g; ++deg) gen_bits[deg] = generator.getCoef(deg);
15     PackedBit gen_packed(gen_bits);
16
17     // 3. Division polinómica: si MSB = 1, XOR del generador alineado
18     for (int i = buffer.size() - 1; i >= g; --i) {
19         if (binaryBuffer.get(i)) {
20             size_t shift = (i - g);
21             binaryBuffer.xorShifted(gen_packed, shift);
22         }
23     }
24
25     // 4. Extraer resto y formar palabra sistemática
26     buffer = binaryBuffer.toVector();
27     std::vector<uint16_t> parity(buffer.begin(), buffer.begin() + g);
28     parity.insert(parity.end(), message.begin(), message.end());
29     return parity;
30 }

```

4.4.4. Detección de errores mediante síndromes

Para el proceso de decodificación, el primer paso consiste en calcular el vector de síndromes. Si todos son cero, el bloque recibido es válido; en caso contrario, se detecta la presencia de errores.

```

1  std::vector<uint16_t> BCH_Codec::syndrome(const std::vector<uint16_t> &received, bool &error) {
2      Polynomial r(gf, received);
3      std::vector<uint16_t> synd(2 * t + 1, 0);
4      error = false;
5      for(int i=1; i<=2*t; i++){
6          if (i%2 == 0){
7              synd[i] = gf.power(synd[i/2],2);

```

```

8         } else{
9             uint16_t root = gf.power(2, i);
10            synd[i] = r.evaluate(root);
11        }
12        if (synd[i] != 0 and !error){
13            error = true;
14        }
15    }
16    return synd;
17 }

```

La implementación aprovecha la propiedad de los síndromes en cuerpos binarios: los índices pares se obtienen por cuadrado del síndrome anterior, lo que reduce el número de evaluaciones directas del polinomio recibido.

4.4.5. Localización de errores con Berlekamp-Massey

Una vez detectada la presencia de errores, la clase estima el polinomio localizador de errores mediante el algoritmo de Berlekamp-Massey. La implementación evita el coste de construir objetos `Polynomial` en cada iteración y trabaja directamente sobre vectores de coeficientes.

```

1 Polynomial BCH_Codec::berlekampMassey(const std::vector<uint16_t>& synd) {
2     std::vector<uint16_t> C = {1};
3     std::vector<uint16_t> B = {1};
4     int L = 0;
5     int shift_m = 1;
6     uint16_t db = 1;
7
8     for (int i = 1; i <= 2 * t; i++) {
9         uint16_t d = synd[i];
10        int max_j = std::min(L, (int)C.size() - 1);
11        for (int j = 1; j <= max_j; j++) {
12            d = gf.add(d, gf.multiply(C[j], synd[i - j]));
13        }
14
15        if (d == 0) {
16            shift_m++;
17        } else {
18            std::vector<uint16_t> oldC = C;
19            uint16_t factor = gf.multiply(d, gf.inverse(db));
20
21            size_t neededSize = B.size() + shift_m;
22            if (C.size() < neededSize) {
23                C.resize(neededSize, 0);
24            }
25
26            for (size_t j = 0; j < B.size(); j++) {
27                C[j + shift_m] = gf.add(C[j + shift_m], gf.multiply(factor, B[j]));
28            }
29
30            if (2 * L <= i - 1) {
31                L = i - L;
32                B = oldC;
33                db = d;

```

```

34         shift_m = 1;
35     } else {
36         shift_m++;
37     }
38 }
39 }
40 return Polynomial(gf, C);
41 }

```

4.4.6. Decodificación

La decodificación combina la detección de errores, la estimación del polinomio localizador y la búsqueda de raíces de ese polinomio mediante la búsqueda de Chien.

El flujo de corrección es el siguiente:

1. Se calcula el síndrome del bloque recibido.
2. Si no hay errores, se extrae directamente la parte de información.
3. Si hay errores, se aplica Berlekamp-Massey para obtener el polinomio localizador.
4. Se realiza una búsqueda de raíces para localizar las posiciones erróneas.
5. Si el número de raíces encontradas coincide con el grado del polinomio, se corrigen los bits y se devuelve el mensaje.

```

1  bool BCH_Codec::decode(const std::vector<uint16_t>& received, std::vector<uint16_t>&
    message_out) {
2      if (received.size() != static_cast<size_t>(n)) {
3          throw std::invalid_argument("La longitud de la palabra recibida debe ser exactamente
    n");
4      }
5
6      std::vector<uint16_t> corrected = received;
7      bool errorsDetected = false;
8
9      // 1. Calcular síndromes para detectar errores del canal
10     std::vector<uint16_t> synd = syndrome(corrected, errorsDetected);
11
12     // Si no hay errores, extraer el mensaje y devolver operacion exitosa
13     if (!errorsDetected) {
14         message_out = std::vector<uint16_t>(corrected.end() - k, corrected.end());
15         return true;
16     }
17
18     // 2. Si hay errores, aplicar Berlekamp-Massey
19     Polynomial errorLocator = berlekampMassey(synd);
20     int L = errorLocator.getDegree();
21
22     // 3. Comprobacion de errores incorregibles
23     if (L == 0 || L > t) {
24         // Fallo: demasiados errores. Devolver mensaje con errores residuales
25         message_out = std::vector<uint16_t>(corrected.end() - k, corrected.end());

```

```

26     return false;
27 }
28
29 // 4. Búsqueda de Chien para encontrar posiciones erroneas
30 std::vector<uint16_t> reg(L + 1, 0);
31 for (int i = 0; i <= L; i++) {
32     reg[i] = errorLocator.getCoef(i);
33 }
34
35 std::vector<uint16_t> factors(L + 1);
36 for (int i = 0; i <= L; i++) {
37     factors[i] = gf.power(2, n - i); // Equivalente a  $\alpha^{-j}$ 
38 }
39
40 int rootsFound = 0;
41 for (int i = 0; i < n; i++) {
42     uint16_t sum = 0;
43     for (int j = 0; j <= L; j++) {
44         sum = gf.add(sum, reg[j]);
45     }
46
47     if (sum == 0) {
48         corrected[i] ^= 1; // Invertir el bit erroneo
49         rootsFound++;
50     }
51
52     // Actualizar registros para la siguiente iteracion
53     for (int j = 0; j <= L; j++) {
54         reg[j] = gf.multiply(reg[j], factors[j]);
55     }
56 }
57
58 // 5. Comprobacion de integridad de la busqueda de Chien
59 if (rootsFound != L) {
60     // Fallo: el polinomio indicaba L errores, pero se encontraron un numero distinto
61     message_out = std::vector<uint16_t>(corrected.end() - k, corrected.end());
62     return false;
63 }
64
65 // 6. Decodificacion exitosa (BCH corrigio el bloque)
66 message_out = std::vector<uint16_t>(corrected.end() - k, corrected.end());
67 return true;
68 }

```

4.4.7. Comentarios de diseño

- **Rendimiento:** PackedBit reduce el número de operaciones XOR.
- **Reutilización:** El cálculo del generador y la decodificación están encapsulados en métodos privados y públicos separados, lo que hace más legible el flujo de la clase.

4.5. Implementación del Canal de Ruido

El componente `Channel` modela los principales canales de ruido usados en las simulaciones: el canal binario simétrico (BSC) y el canal AWGN con decisión dura. La clase encapsula un generador de números aleatorios `std::mt19937` y expone métodos claros para aplicar cada modelo de ruido.

4.5.1. Interfaz pública

La interfaz de `Channel` se resume en:

```
1 class Channel{
2 private:
3     std::mt19937 rng; // Generador de numeros aleatorios
4 public:
5     Channel();
6     std::vector<uint16_t> applyBSC(const std::vector<uint16_t>& bits, double bit_error_rate);
7     std::vector<double> transmitAWGN(const std::vector<uint16_t>& bits, double es_n0_db);
8     std::vector<uint16_t> applyAWGNHardDecision(const std::vector<uint16_t>& bits, double
        es_n0_db);
9 };
```

4.5.2. Constructor

El constructor inicializa `rng` con una semilla no determinista tomada de `std::random_device` para obtener trayectorias de ruido reproducibles solo si se sustituye la semilla.

```
1 Channel::Channel() {
2     std::random_device rd;
3     rng = std::mt19937(rd());
4 }
```

4.5.3. Canal Binario Simétrico (BSC)

`applyBSC` aplica fallos independientes por bit con probabilidad `bit_error_rate` usando una distribución de Bernoulli. Valida el parámetro de probabilidad y lanza `std::invalid_argument` si está fuera de rango.

```
1 std::vector<uint16_t> Channel::applyBSC(const std::vector<uint16_t>& bits, double
    bit_error_rate) {
2     if (bit_error_rate < 0.0 || bit_error_rate > 1.0) {
3         throw std::invalid_argument("bit_error_rate must be between 0 and 1");
4     }
5
6     std::bernoulli_distribution flip(bit_error_rate);
7     std::vector<uint16_t> noisy_bits = bits;
8     for (uint16_t& bit : noisy_bits) {
9         if (flip(rng)) bit ^= 1;
```

```

10     }
11
12     return noisy_bits;
13 }

```

4.5.4. Canal AWGN

Para señalización binaria antipodal se mapea cada bit en símbolo $+1$ o -1 y se añade ruido Gaussiano con desviación estándar calculada a partir de E_s/N_0 . La función auxiliar `dbToLinear` convierte dB a escala lineal.

```

1  double dbToLinear(double db_value) {
2      return std::pow(10.0, db_value / 10.0);
3  }
4
5  std::vector<double> Channel::transmitAWGN(const std::vector<uint16_t>& bits, double es_n0_db) {
6      const double es_n0_linear = dbToLinear(es_n0_db);
7      const double sigma = std::sqrt(1.0 / (2.0 * es_n0_linear));
8
9      std::normal_distribution<double> noise(0.0, sigma);
10     std::vector<double> samples; samples.reserve(bits.size());
11
12     for (uint16_t bit : bits) {
13         const double symbol = (bit == 0) ? 1.0 : -1.0;
14         samples.push_back(symbol + noise(rng));
15     }
16
17     return samples;
18 }

```

4.5.5. Decisión dura sobre AWGN

La función `applyAWGNHardDecision` realiza la transmisión AWGN y aplica una decisión dura: si la muestra es ≥ 0 se interpreta como 0, en caso contrario como 1.

```

1  std::vector<uint16_t> Channel::applyAWGNHardDecision(const std::vector<uint16_t>& bits, double
    es_n0_db) {
2      std::vector<double> samples = transmitAWGN(bits, es_n0_db);
3      std::vector<uint16_t> hard_bits; hard_bits.reserve(samples.size());
4      for (double sample : samples) hard_bits.push_back(sample >= 0.0 ? 0u : 1u);
5      return hard_bits;
6  }

```

4.5.6. Decisiones de diseño y uso

- **Separación de responsabilidades:** el canal ofrece modelos de canal (BSC, AWGN) como funciones independientes, facilitando su comparación experimental.

- **Reproducibilidad:** la semilla de `rng` se extrae de `random_device` por defecto; para tests reproducibles puede sustituirse por una semilla fija.

4.6. Arquitectura del Simulador y Motor de Ejecución

La clase `BCH_Simulator` actúa como el motor principal del proyecto. Su responsabilidad es orquestar las instancias del codificador/decodificador (`BCH_Codec`) y del canal ruidoso (`Channel`), gestionar el barrido de relaciones señal a ruido (E_b/N_0), y recolectar las métricas de rendimiento para su posterior análisis.

4.6.1. Estructuras de Configuración y Resultados

Para garantizar la modularidad, los parámetros de entrada se encapsulan en la estructura `SimulationConfig`, la cual define los parámetros del código (m, t), el rango de simulación, y los criterios de parada estadística. De forma análoga, la estructura `PointResults` consolida las métricas de salida por cada punto de E_b/N_0 .

```

1  struct PointResults {
2      double ebno_db;
3      double ber;           // Bit Error Rate (Codificado)
4      double ber_uncoded;   // Bit Error Rate (Sin codificar)
5      double cwer;         // Codeword Error Rate
6      long codewords_simulated;
7      double avg_enc_us;    // Latencia media de codificación
8      double avg_dec_us;    // Latencia media de decodificación
9  };
10
11 struct SimulationConfig {
12     int m = 7;
13     int t = 10;
14     double ebno_min = 0.0;
15     double ebno_max = 12.0;
16     double step = 1.0;
17     int min_codeword_errors = 100; // Criterio de parada Monte Carlo
18     long max_codewords = 1000000;
19     bool use_file = false;
20     std::string input_path = "";
21 };

```

4.6.2. Motor de Simulación y Paralelización (OpenMP)

El núcleo computacional del simulador reside en el método `simulatePoint`, encargado de generar las transmisiones para un valor específico de E_b/N_0 . Dado que las simulaciones de Monte Carlo para tasas de error bajas ($\text{BER} < 10^{-5}$) requieren procesar millones de bloques, este método ha sido paralelizado utilizando OpenMP.

Gestión de Concurrencia y Condición de Parada

Para maximizar el escalado multinúcleo, se instancian copias locales del codificador y del canal por cada hilo (*thread*), eliminando cualquier contención por memoria compartida. Asimismo, la recolección de estadísticas globales se realiza mediante `#pragma omp atomic`, las cuales son operaciones a nivel de hardware significativamente más rápidas que los bloques `critical` tradicionales.

Un aspecto fundamental de este motor es su criterio de parada dinámico. En lugar de procesar siempre el límite máximo de bloques (`max_codewords`), los hilos monitorizan de forma atómica el número de tramas erróneas acumuladas. Una vez superado el umbral de significancia estadística (`min_codeword_errors`, típicamente 100 errores), los hilos abortan el procesamiento de nuevos bloques. Esta decisión reduce drásticamente el tiempo de simulación en canales ruidosos (valores bajos de E_b/N_0), concentrando la potencia de cálculo únicamente en las regiones asintóticas donde los errores son infrecuentes.

```

1 PointResults BCH_Simulator::simulatePoint(...) {
2     // ... (Inicialización de variables y conversión a Es/NO) ...
3
4     #pragma omp parallel
5     {
6         std::mt19937 rng(std::random_device{}()); // Generador PRNG aislado por hilo
7         BCH_Codec local_bch = bch;
8         Channel local_channel = channel;
9
10        #pragma omp for schedule(dynamic)
11        for (long iter = 0; iter < max_iterations; ++iter) {
12
13            // Condicion de parada estadística (Monte Carlo)
14            if (!cfg.use_file) {
15                long cur_codeword_errors;
16                #pragma omp atomic read
17                cur_codeword_errors = total_codeword_errors;
18                if (cur_codeword_errors >= cfg.min_codeword_errors) continue;
19            }
20
21            // ... (Generación, Codificación y Transmisión) ...
22
23            // Evaluación del éxito de la decodificación
24            if (!success || bit_errors_in_codeword > 0) {
25                #pragma omp atomic
26                total_codeword_errors++;
27                #pragma omp atomic
28                total_bit_errors += bit_errors_in_codeword;
29            }
30
31            #pragma omp atomic
32            total_bit_errors_uncoded += bit_errors_uncoded_in_codeword;
33
34            #pragma omp atomic
35            codewords++;
36        }
37    }
38    // ... (Cálculo de tasas finales BER y CWER) ...
39 }

```

4.6.3. Decisiones de Diseño Adicionales

- **Inyección de Tráfico Real:** Además del generador pseudoaleatorio, el simulador permite inyectar archivos binarios reales (*files*) como fuente de datos. Esto no solo valida la total transparencia del sistema ante cualquier secuencia de bits, sino que permite extraer archivos corruptos post-simulación para analizar cualitativamente el impacto de los errores residuales (por ejemplo, evaluando la degradación visual en archivos de imagen).
- **Métricas de Latencia:** La recolección de los tiempos de codificación y decodificación (`avg_enc_us`, `avg_dec_us`) se realiza mediante la librería `<chrono>` de alta resolución, encapsulando estrictamente las llamadas algorítmicas para aislar el coste real del *hardware* frente a operaciones de E/S de memoria.
- **Exportación Transparente:** Todas las métricas convergidas se vuelcan automáticamente en formato `CSV`, permitiendo su ingesta directa en herramientas de post-procesado como Python (Matplotlib/Pandas) para la generación de las curvas de rendimiento abordadas en el siguiente capítulo.

Capítulo 5

Resultados y Análisis

En este capítulo se presentan y analizan los resultados obtenidos con el simulador BCH desarrollado. Se examina el rendimiento del código en la corrección de errores (BER y CWER) y se evalúa la eficiencia computacional, siguiendo dos enfoques complementarios.

5.1. Metodología Experimental

Para evaluar el rendimiento, se ha implementado un entorno de simulación basado en el método de Montecarlo, el cual sigue un flujo de: generación de información, codificación, simulación del canal ruidoso y decodificación.

Se han aplicado los siguientes criterios para garantizar la validez estadística de las curvas de rendimiento obtenidas:

- **Modelo de canal:** Los bits de cada palabra $c_i \in \{0, 1\}$ se mapean a valores en el rango $s_i \in \{+1, -1\}$ (BPSK). El canal se modela como un canal de ruido blanco gaussiano aditivo (AWGN) con decisión dura. La varianza del ruido generado (σ^2) se calcula dinámicamente para cada punto de simulación a partir de la relación señal a ruido por símbolo (E_s/N_0), la cual se deriva de E_b/N_0 mediante la tasa de código $R = k/n$, siguiendo la relación:

$$\frac{E_s}{N_0} = \frac{E_b}{N_0} \cdot R$$

A partir de este valor, la desviación típica del ruido viene dada por:

$$\sigma = \sqrt{\frac{1}{2 \cdot \frac{E_s}{N_0}}}$$

- **Criterios de parada:** Con el objetivo de evitar estimaciones imprecisas en regiones de alta relación señal a ruido (donde los eventos de error son muy poco probables), el simulador incorpora un doble criterio de parada para cada punto de E_b/N_0 :

1. Se define un volumen objetivo de procesamiento de hasta 2×10^9 bits de información para estimar de forma fiable tasas de error extremadamente bajas.
 2. Para optimizar el tiempo de cómputo, el número total de palabras de código a simular para cada punto de E_b/N_0 se calcula como $\text{max_codewords} = \text{target_bits}/n$. Este valor se encuentra acotado inferiormente en 50.000 bloques (para garantizar una muestra mínima en códigos cortos con valores de m bajos) y superiormente en 2.000.000 de bloques (para mitigar el impacto del crecimiento exponencial del tiempo de cómputo en configuraciones con $m = 15$ y alta capacidad correctora t).
 3. Se exige un umbral mínimo de 100 palabras de código erróneas para dar por concluida la evaluación de un punto específico de la curva. Si se alcanza dicho umbral de errores antes de agotar el máximo de bloques configurado, el punto se da por finalizado. Por el contrario, si la CWER desciende a 0 de forma absoluta en un punto determinado, la ejecución de la curva se interrumpe al alcanzarse el límite de corrección del sistema.
- **Entorno de ejecución y paralelización:** Debido a la alta carga computacional del procesamiento de miles de millones de bits, el núcleo del simulador se ha paralelizado mediante OpenMP utilizando *dynamic schedule*, lo que permite distribuir la ejecución de los bucles de Montecarlo de manera equilibrada entre los hilos del procesador, aislando los generadores de números aleatorios y las configuraciones del codec de manera local para cada hilo con el fin de evitar condiciones de carrera.

5.2. Impacto de la capacidad de corrección t para un valor de m fijo

En esta sección se analiza el comportamiento del código BCH cuando se mantiene fija la longitud del bloque y se varía su capacidad de corrección de errores (t). Se ha seleccionado el cuerpo de grado $m = 7$, que corresponde a un Cuerpo de Galois $GF(2^7)$ y una longitud de bloque fija de $n = 2^7 - 1 = 127$ bits.

Se han evaluado las configuraciones $t \in \{1, 3, 7, 15\}$. La Tabla 5.1 muestra los parámetros principales de cada código, incluida su tasa ($R = k/n$).

Tabla 5.1: Parámetros de los códigos BCH para $m = 7$ ($n = 127$).

Capacidad (t)	Dimensión (k)	Redundancia	Tasa (R)
1	120	7	0,945
3	106	21	0,835
7	78	49	0,614
15	22	105	0,173

A partir de los resultados mostrados en la Figura 5.1, se pueden identificar tres comportamientos principales:

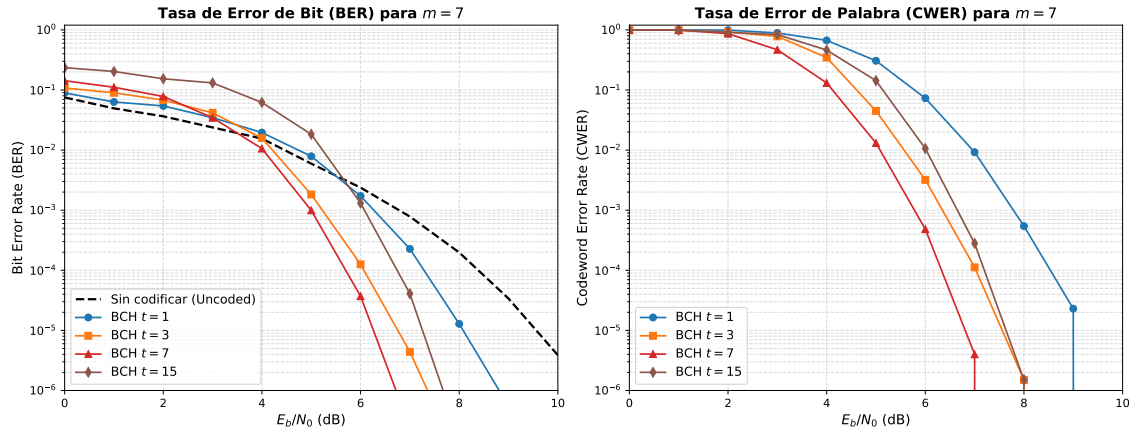


Figura 5.1: Estudio comparativo para $m = 7$: Tasa de Error de Bit (BER, izquierda) y Tasa de Error de Palabra (CWER, derecha) para $t \in \{1, 3, 7, 15\}$.

5.2.1. Saturación del decodificador en entornos de alto ruido

En condiciones de ruido elevado (aproximadamente entre 0 y 3 dB), se observa un comportamiento contraintuitivo: el código con mayor capacidad teórica de corrección ($t = 15$) presenta la peor tasa de error de bit (BER).

Esto ocurre porque su baja tasa de código ($R = 0,173$) obliga al canal a introducir un volumen mucho mayor de errores para mantener la misma energía por bit de información (E_b/N_0). Cuando el número de errores supera la capacidad de corrección, el algoritmo de Berlekamp-Massey genera un polinomio localizador incorrecto. El decodificador intenta entonces corregir posiciones erróneas, lo que añade más errores al bloque. Como consecuencia, la tasa de error de palabra (CWER) se acerca a 1 y la BER llega a ser incluso superior a la del canal sin codificar.

5.2.2. Efecto umbral y coste de la redundancia

A partir de unos 4 dB, las curvas muestran el típico «efecto umbral» de la decodificación algebraica con decisión dura. Sin embargo, un mayor valor de t no siempre implica un mejor rendimiento.

La configuración $t = 7$ ofrece el mejor comportamiento global: presenta la caída más rápida de la BER y desplaza el punto de convergencia hacia valores más bajos de E_b/N_0 . Por el contrario, el código con $t = 15$ mejora de forma más lenta y su curva está desplazada hacia la derecha respecto a $t = 3$ y $t = 7$.

Este resultado pone de manifiesto la **penalización por la tasa de código**. Aunque $t = 15$ tiene mayor capacidad correctora, su elevada redundancia hace que el canal sea mucho más hostil, superando con frecuencia la capacidad real del decodificador.

5.2.3. Diferencia entre BER y CWER

La comparación de ambas gráficas muestra que la tasa de error de palabra (CWER) siempre desciende más lentamente que la tasa de error de bit (BER). Esto es lógico, ya que la CWER considera que un bloque completo ha fallado aunque solo quede un único bit erróneo, mientras que la BER mide el promedio de bits erróneos sobre el total de bits transmitidos.

5.2.4. Conclusión

Los resultados demuestran que el rendimiento del código no mejora de forma monótona al aumentar la capacidad de corrección t . Para $m = 7$, existe un punto de equilibrio óptimo en torno a $t = 7$, que ofrece una buena protección sin incurrir en la fuerte penalización que sufre $t = 15$, ni quedarse corto como ocurre con $t = 1$ y $t = 3$.

Este análisis confirma que el diseño de un códec BCH eficaz requiere equilibrar adecuadamente la distancia de corrección con la tasa de código y la penalización energética asociada.

5.3. Comparación de rendimiento entre códigos con tasa de código similar

En esta sección se aborda la evaluación del rendimiento aislando el efecto de la penalización por tasa de código (*code rate penalty*). Al comparar configuraciones con diferentes grados de extensión (m) pero que comparten una tasa de código ($R = k/n$) equivalente, la varianza del ruido en el canal físico (σ^2) se mantiene constante para cada valor de E_b/N_0 evaluado. Este diseño experimental permite analizar de forma directa el impacto de la longitud del bloque (n) y de la capacidad correctora (t) sobre la mitigación de errores.

A partir de los datos obtenidos, se han estructurado tres escenarios de evaluación (tasa alta, media y baja) basados en los grados de extensión $m \in \{5, 7, 11\}$, cuyos parámetros constructivos se detallan en la Tabla 5.2.

Tabla 5.2: Agrupación simétrica de códigos BCH bajo criterios de tasa de código similar.

Escenario	Grado (m)	Bloque (n)	Capacidad (t)	Información (k)	Tasa (R)
Tasa Alta ($R \approx 0,82$)	5	31	1	26	0,838
	7	127	3	106	0,835
	11	2047	35	1662	0,812
Tasa Media ($R \approx 0,50$)	5	31	3	16	0,516
	7	127	9	64	0,504
	11	2047	93	1024	0,500
Tasa Baja ($R \approx 0,34$)	5	31	5	11	0,355
	7	127	14	43	0,338
	11	2047	124	683	0,334

5.3.1. Análisis del rendimiento en función de la longitud de bloque

El primer enfoque de este estudio se presenta en la Figura 5.2, donde las curvas de rendimiento se agrupan manteniendo una tasa de código similar en cada gráfica. De este modo, es posible observar la influencia intrínseca de la longitud del bloque sobre el comportamiento del sistema frente al ruido.

A partir de los resultados experimentales, se identifican dos tendencias fundamentales comunes a los tres escenarios evaluados:

- **Efecto umbral y caída en cascada (*waterfall*) en bloques largos:** En todos los niveles de tasa de código, las configuraciones con bloques de mayor longitud ($m = 11$) exhiben una pendiente de caída de la BER considerablemente más pronunciada. Aunque en regímenes de ruido intenso (valores bajos de E_b/N_0) su rendimiento es similar o ligeramente inferior al de los bloques cortos debido a la propagación de errores en ráfaga, una vez se supera un determinado umbral de relación señal-ruido, la BER disminuye drásticamente. Esto confirma que los códigos largos aprovechan con mayor eficiencia la redundancia conforme mejora el canal.
- **Robustez inicial y ganancia progresiva en bloques cortos:** Por el contrario, las configuraciones cortas ($m = 5$) muestran un comportamiento más lineal y estable en escenarios de baja relación señal-ruido, degradándose de forma menos abrupta. Sin embargo, al carecer de la pronunciada pendiente de caída característica de los códigos de gran longitud, estas configuraciones cortas requieren valores de E_b/N_0 notablemente más altos para alcanzar tasas de error muy bajas (por debajo de 10^{-4}).

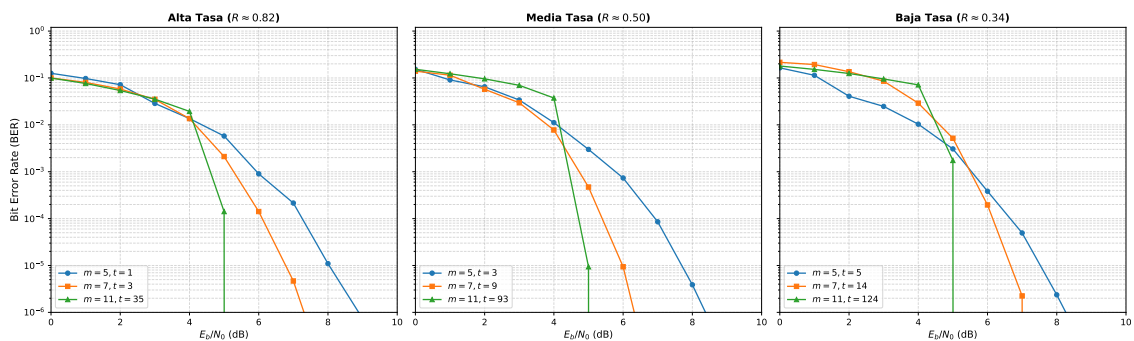


Figura 5.2: Estudio comparativo de la BER para agrupaciones de tasa similar: escenarios de tasa alta (izquierda), media (centro) y baja (derecha).

5.3.2. Análisis de sensibilidad según la capacidad correctora

Para complementar el estudio previo, la Figura 5.3 agrupa las curvas de rendimiento fijando el grado de extensión m en cada subgráfica. Este enfoque permite aislar el impacto que produce el incremento de la capacidad correctora (t) —y la consecuente reducción de la tasa de código (R)— dentro de un mismo tamaño de bloque.

Los resultados experimentales demuestran que añadir redundancia no produce el mismo efecto en todos los horizontes de longitud de código:

- **Bloques cortos ($m = 5$):** En este régimen, las tres configuraciones presentan un comportamiento prácticamente idéntico en la zona de relación señal-ruido baja y media (entre 0 y 6 dB), mostrando curvas que se solapan casi por completo. Esto evidencia que, para bloques tan cortos, incrementar la capacidad correctora de $t = 1$ a $t = 5$ apenas aporta una ganancia perceptible en la BER. Es a partir de los 6 dB donde la configuración de alta tasa ($t = 1$, curva azul) comienza a degradarse y a distanciarse negativamente de las otras dos opciones. Por su parte, las curvas de tasa media ($t = 3$) y baja ($t = 5$) se mantienen pegadas en todo el recorrido; por tanto, la configuración intermedia ($t = 3$) resulta la más eficiente a efectos prácticos, ya que logra el mismo límite de error que la opción con máxima redundancia pero sin penalizar la velocidad de transmisión de información útil.
- **Bloques intermedios ($m = 7$):** Aquí emerge un comportamiento no lineal respecto a la redundancia. La configuración de tasa media ($t = 9$, curva naranja) ofrece el rendimiento óptimo del panel, exhibiendo la caída de BER más rápida. Por el contrario, saturar el bloque con mayor capacidad correctora ($t = 14$, curva verde) resulta contraproducente: presenta el peor desempeño de la zona con ruido intenso (hasta los 6 dB) debido a la severa penalización de la tasa de código, y solo iguala a la configuración intermedia en el extremo de alta relación señal-ruido.
- **Bloques largos ($m = 11$):** Para códigos de gran longitud, el impacto de la penalización por tasa de código en entornos hostiles es muy evidente. En la zona de baja relación señal-ruido (de 0 a 4 dB), la configuración con máxima redundancia ($t = 124$, curva verde) es superada de forma clara por las tasas media y alta, posicionándose como la opción con peor rendimiento. No obstante, en el intervalo entre los 4 y los 5 dB, esta tendencia cambia de manera drástica: la curva verde experimenta una caída en cascada vertical e instantánea. Al venir desde una zona de errores más elevada, protagoniza el descenso más significativo del panel, logrando que la BER caiga por debajo de 10^{-6} y converja en el límite de error cero en el mismo punto que las demás configuraciones en cuanto el canal empieza a despejarse.

5.3.3. Discusión global y criterios de diseño

Los resultados obtenidos en los diferentes escenarios evidencian que el beneficio de incrementar la redundancia está condicionado por la longitud del bloque del código. En estructuras de bloques cortos ($m = 5$), las curvas se solapan y añadir más paridad no aporta una ganancia apreciable frente al ruido. En cambio, en bloques intermedios ($m = 7$) y largos ($m = 11$), un exceso de bits de paridad penaliza severamente el rendimiento en canales degradados, requiriendo superar un umbral crítico de relación señal-ruido para activar su potencial de corrección en cascada.

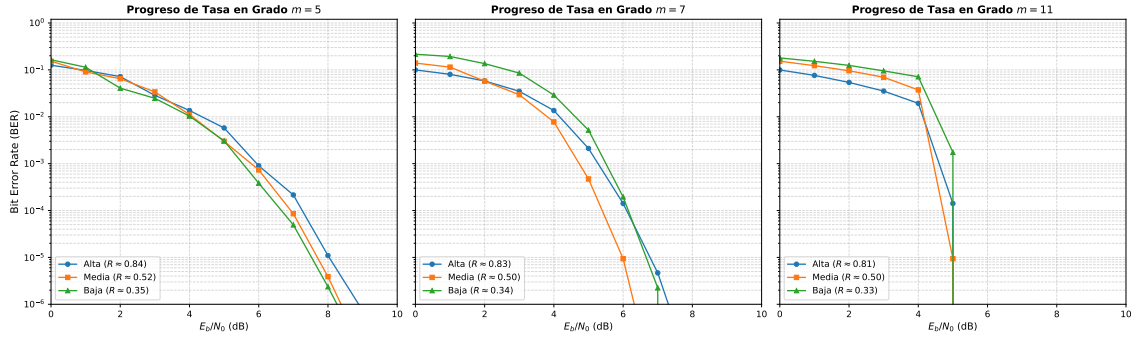


Figura 5.3: Estudio de sensibilidad de la tasa de código agrupado por grado de extensión: $m = 5$ (izquierda), $m = 7$ (centro) y $m = 11$ (derecha).

En consecuencia, las simulaciones certifican la inexistencia de una solución universal y permiten definir criterios de diseño claros: si el sistema opera con restricciones de tamaño de bloque corto, la configuración de tasa media representa el compromiso óptimo, al igualar el rendimiento de la tasa baja sin penalizar la velocidad de transmisión. Por el contrario, si se implementan bloques largos en canales donde se garantiza superar los 5 dB, las configuraciones de alta tasa son las más eficientes, ya que logran la convergencia a error cero en el mismo umbral que las demás pero maximizando la tasa de transferencia de datos útiles.

Bibliografía